



16. Juni 2026

Übungen zur Vorlesung Service-basierte Komponenten-Architekturen

SS 2026

Übungsblatt Nr. 6

(Abgabe: Bearbeitung bis Dienstag, 30.06.2026)

Anmerkung:

Nutzen Sie die nächsten Wochen, um an Ihren Semesterprojekten weiterzuarbeiten. Dazu stelle ich Ihnen zwei Aufgaben bereit, die Sie als Basis gerne verwenden können. Nutzen Sie die nächsten zwei Wochen, um einen finalen Stand oder aber zumindest einen Arbeitsstand zu präsentieren. Fokussieren Sie sich dabei auf die Aufgabe 1 oder 2.

In der übernächsten Woche (30.6.2026) wird das dritte Semesterprojekt im Rahmen der **Übung Nr. 7** eingeführt, welches Sie natürlich auch bearbeiten können.

Am 14.7.2026 werden durch die **Übung Nr. 8** die finalen (ergänzenden) Anforderungen für die verschiedenen Semesterprojekte vermittelt.

Aufgabe 1 (Weiter-Entwicklung der Software für das Unternehmen WirSchiffenDas; Bearbeitung bis: 30.6.2026)

Die Geschäftsleitung der Firma WirSchiffenDas GmbH ist angetan von den neuen Technologien aus den letzten Kapiteln der Vorlesung, d.h., Docker, Apache Kafka, und den Spring-Netflix-Stack! Es soll Ihre Aufgabe sein, *mindestens* einer dieser Technologien sinnvoll in Ihre Anwendung für die Fallstudie der Firma zu integrieren. Konkret sollten Sie *mindestens eine* der folgenden technischen Anforderungen umsetzen:

MS_TA1: Verwendung des Framework Apache Kafka für eine Integration der Microservices zur Übermittlung insbesondere der „Bearbeitungs-Status-Nachrichten“.

MS_TA2: Verwendung der Technologien Docker und Docker Compose zur Bereitstellung und Komposition der Microservices in einsetzbare Docker Container.

MS_TA3: Verwendung des Netflix-Stacks, um wichtige Pattern wie Service Registry oder Circuit Breaker sinnvoll zu integrieren (Alternative: Resilience4j oder Spring Cloud)

MS_TA4: Sinnvolle Integration einer Technik zur Orchestration von Services (Camunda oder MCP)

a.)

Erweitern sie ihren bisherigen Prototypen entsprechend. Eine vollständige Implementierung bis zum 30.6.26 ist *nicht* erwartet. Wenn Sie dieses Thema als Semesterprojekt verwenden, dann sollten Sie dann eine möglichst vollständige Implementierung demonstrieren können.

b.)

Modellieren sie zudem die resultierende Software-Architektur mit Hilfe des 4-Sichten-Modells! Verwenden sie auch eine Farben-Legende, um Anpassungen oder Erweiterungen an ihrer Architektur gegenüber der Architektur aus der Übung Nr. 5 (siehe Hinweise in Kapitel 3, Abschnitt 2) hervorzuheben. Auch die Quelle von verwendeten Technologien (z.B. COTS, OSS) können sie optional mit einer Farben-Legende versehen. Bei den Technologien aus Kapitel 5 ggf. vorarbeiten.

c.)

Betrachten Sie auch noch mal den Katalog nach (Schirgi and Brenner, 2021). Priorisieren sie noch mal die Lösungen für Anti Pattern bzw. Architecture Smells, die sie für ihr Konzept bzw. für ihren Prototyp als relevant ansehen. In einer möglichen Präsentation sollten sie die vier höchst priorisierten Lösungen gemäß ihrer Bewertung vorführen und erläutern können. Es sollten für das finale Semesterprojekt *mindestens zwei* Lösungen in ihr Konzept bzw. in den Prototyp einfließen.

Hinweise für die Verwendung von Kafka: Verwenden sie die Installation aus SEPP v 2.1:

<https://saschaalda.wordpress.com/sepp-software-engineering-deployment-pipeline-hbrs/>

Einen aktuellen Überblick zu SEPP erhalten Sie hier:

<https://github.com/LUC4R4T0R/SEPP-diagrams/blob/main/exports/Overview/Overview.png>

Verwenden sie als Topic eine möglichst individuelle Bezeichnung, um mögliche Kollisionen zu vermeiden, z.B. `mytopic_myname`.

Die in der Vorlesung bzw. in dem Kapitel 4 gezeigten Source Codes finden sie unter:

Kafka-Demo (kein SpringBoot)

<https://git.fslab.de/salda2m/kafkademo>

Stream-Application:

<https://git.fslab.de/salda2m/kafkastreamapp>

Aufgabe Nr. 2 (Orchestration von Services für das Order Fullfillment mit Camunda und MCP; Freiwillige, erste Bearbeitung bis 30.6.2026)

In der Vorlesung haben Sie im Kapitel Nr. 6 grundlegende Möglichkeiten für die Orchestration von Services kennengelernt. Sie können als Alternative zu der Aufgabe Nr. 1 den Prozess „Order Fullfillment“ auf Basis von Camunda entwickeln, mit dem Sie eine traditionelle, manuelle Orchestration basierend auf einem Workflow-Modell umsetzen. Den Workflow sollten Sie mit der Notation BPMN 2.0 modellieren:

- FA1: Manuelle Orchestration von (Micro-)Services für den Prozess „Order Fullfillment“ mit Camunda (inklusive einer *einheitlichen* Web-UI)
 - Auch hier sollten mindestens eine der technischen Anforderungen *MS_TA2* und *MS_TA3* umgesetzt werden
 - Die einzelnen Services des Order-Fullfillments (Shipping of Products, Billing of Products, Managing of Customer) können Sie als Microservices implementieren und mit Beispieldaten füllen. Hinweise für die API der Services finden Sie auf Fol. 9 der Beschreibung der Fallstudie.
 - Als Web-UI können Sie ein eigenes UI auf Basis eines Frameworks wie z.B. Angular oder Vaadin entwickeln. Diese greifen dann auf die REST-API des Camunda Engine zu, um z.B. Workflow zu initiieren. Alternativ können Sie die Eingaben und Ausgaben über Camunda Forms realisieren, die dann in dem Camunda Cockpit aufgerufen werden können.
 - Bei dieser Variante empfehle ich die Verwendung der Version 7, die Sie lokal auf Ihrem Rechner installieren können.

Als Alternative dazu können Sie die Orchestration über eine LLM-kompatible Anwendung (z.B. ChatGPT, Claude) steuern, bei dem die Microservices automatisiert orchestriert werden. Auch Camunda kann hier als LLM-kompatible Anwendung (sog. „AI Agent“) fungieren. Damit ergeben sich zwei folgende Alternativen:

- FA2a: Automatisierte Orchestration von (Micro-)Services für den Prozess „Order Fullfillment“ mit MCP und einer LLM (inklusive einer CLI / Chat).
 - Hierzu müssen die Microservices über das MCP-Protokoll als MCP-Server bereitgestellt werden und in einem KI-Agent wie z.B. Claude registriert werden.
 - Der Chat kann dann über ein Tool wie Claude verwendet werden.
 - Bei dieser Variante empfehle ich Sprache Python mit dem Framework Fast-MCP: <https://gofastmcp.com/getting-started/welcome>
- FA2b: Automatisierte Orchestration von (Micro-)Services für den Prozess „Order Fullfillment“ mit Camunda im Rahmen eines AI-Agents (Ad-hoc Teilprozess).
 - Hierzu müssen die Microservices über Rest Outbound Connector sowie mit der Sprache FEEL in Camunda registriert werden.
 - Der Chat bzw. die UI kann dann als Camunda Form bereitgestellt werden.
 - Bei dieser Variante sollte die Camunda Version 8 (z.B. Self-Managed) verwendet werden.

